

DECISION LOG

Skylark Drones BI Agent

AI Engineer Technical Assignment | March 2026

What is this document: A Decision Log records every major technical choice made during development — what options were considered, what was chosen, and why. It demonstrates production-level thinking and intentional engineering.

1. Project Overview

The Skylark Drones BI Agent is a live, conversational business intelligence tool that allows founders and senior executives to query their deal pipeline and work order data using natural language. It fetches data directly from Monday.com at query time, builds a structured analytical context, and returns crisp, data-backed answers via a Groq-hosted LLM.

Component	Technology Used
Frontend / UI	Streamlit (Python)
Data Source	Monday.com GraphQL API (live, no cache)
AI Model	Groq — Llama 3.1 8B Instant
Analytics Layer	Pandas + NumPy (Python)
Hosting	Streamlit Cloud
Language	Python 3.11

2. Architecture Overview

The agent follows a strict 5-layer pipeline executed fresh on every query:

Layer	File	Responsibility
1. Data Fetch	data_layer.py	GraphQL API calls to Monday.com — both boards, live every query
2. Normalization	data_layer.py	Parse dates to datetime64, normalize sectors, fix billing typos, treat 0.0 as missing
3. Context Build	context_builder.py	Pre-compute all analytics: win rates, sector stats, velocity, aging, weighted pipeline, quarterly trends
4. Prompt Engine	prompt_engine.py	5-layer PTFC prompt: Domain Knowledge Math Rules Null Safety Format Filter Memory
5. LLM Answer	intelligence_layer.py	Groq API call with structured context JSON, returns answer string

Key design principle: all calculations happen in Python (context_builder.py) before the LLM is called. The LLM receives pre-computed numbers and is only responsible for selecting, formatting, and presenting them — never computing. This prevents hallucination and ensures mathematical accuracy.

3. Monday.com Integration

3.1 How integration works

Every query triggers a fresh GraphQL API call to Monday.com — no caching, no pre-loading. This ensures the founder always sees live data.

Board	Board ID	Records	Fields Fetched
Deals Pipeline	5026905724	344 items	11 columns: status, sector, value, probability, stage, dates
Work Orders	5026903340	176 items	27 columns: financials, execution status, billing, dates

- GraphQL query fetches all items in a single request (limit: 500) with column_values including id, text, and value
- Column IDs are hardcoded constants mapped to human-readable field names (e.g., color_mm10njxv → deal_status)
- API version pinned to 2024-01 for stability
- Timeout set to 15 seconds per board; both boards fetched sequentially

3.2 Live fetching strategy (no caching)

Decision: Fetch live on every query — no @st.cache_data, no session-level preloading

The function fetch_and_normalize_all() is called at the start of every process_question() invocation. The Groq client is cached with @st.cache_resource (connection object only, not data). This means:

- Every answer reflects Monday.com as of the moment the question was asked
- No stale data risk — no TTL management needed
- Typical fetch time: 5-6 seconds per board (10-12s total data fetch)
- Trade-off accepted: slightly slower response time in exchange for guaranteed data freshness

4. Key Technical Decisions

Decision Area	Options Considered	Choice Made	Reason
LLM Provider	OpenAI GPT-4o, Anthropic Claude, Groq Llama	Groq Llama 3.1 8B Instant	Free tier offers 500K tokens/day vs 100K for 70B model. 5x more daily queries. Speed is 2-3x faster. For structured BI answers

			from pre-computed context, 8B performs on par with 70B.
Computation: LLM vs Python	Let LLM compute metrics, Pre-compute in Python and pass to LLM	Pre-compute everything in Python (context_builder.py)	LLMs hallucinate arithmetic. All win rates, collection rates, weighted forecasts are computed in Pandas before the LLM is called. LLM only selects and formats — never calculates.
Date Handling	Return datetime.date objects, Return pd.Timestamp	pd.Timestamp via pd.to_datetime()	Python date objects mixed with NaN floats created a mixed-dtype object column. This caused >= comparison crashes. pd.Timestamp ensures clean datetime64 dtype throughout.
Prompt Architecture	Single flat prompt, Multi-layer structured prompt	5-layer PTFC prompt (Domain Math Null Safety Format Filter Memory)	Each layer addresses a specific failure mode: hallucination (null safety), wrong formulas (math rules), wrong sector groupings (domain layer), poor formatting (format layer), lost filters (filter memory).
Token Efficiency	json.dumps(indent=2) for context, json.dumps() no indent	No indent serialization + compressed system prompt	Indented JSON used 2800 tokens vs 1293 tokens for context. Combined with prompt compression (2368 → 542 tokens), reduced from 5868 to ~2785 tokens per query — doubling daily query capacity.
Session Filter Memory	No filter persistence, Rule-based regex extraction	extract_new_filters() with cumulative filter list in st.session_state	Founders ask follow-up questions like 'now exclude energy' or 'above 50k'. Filters must accumulate across turns. Regex extracts sector/value/time filters and stores them in session state, injecting them into every subsequent prompt.
UI Input	st.text_input + st.button (separate)	st.form with clear_on_submit=True	With separate text_input and button, Enter key did not submit and the input field retained old text after answer. st.form solves both: Enter submits the form and clear_on_submit=True empties the field automatically.
Data Quality	Pass raw data to LLM	Pre-flag data quality issues as quality_summary dict	179 deals (52%) have missing values. 258 (75%) have no closure probability. Rather than letting the LLM discover this mid-answer, quality issues are detected during normalization and surfaced as structured caveats in the UI.

5. Data Quality Handling

The dataset has significant quality issues that were discovered during development and handled systematically:

Issue Found	How Handled	Impact
52% of deals missing deal value	Track count separately; always caveat pipeline totals with 'based on X of Y deals'	Pipeline figures are partial — stated explicitly in every answer
75% missing closure probability	Count separately; weighted forecast uses 10% for unrated deals	Weighted forecast is conservative — acknowledged in methodology field
Duplicate header rows in API response	Filter rows where deal_status == 'Deal Status' during normalization	2 rows removed; would have inflated deal counts
Billing typo: 'Billed' in Monday.com	String replace during normalization; count tracked in quality_summary	3 records fixed; typo count shown in Data Quality Notes expander
Mixed date/float types in columns	All date columns converted to datetime64 via pd.to_datetime(errors='coerce') at start of build_deals_context()	Prevented >= comparison crash between datetime.date and float NaN
Financial zeros (0.0 = missing)	All financial columns: treat 0.0 as None (not actual zero revenue)	Prevents inflating collection rates and contract totals
'This quarter' using wrong date	Use pd.Timestamp(datetime.now()) as 'today', not deals_df.created_date.max()	Max date was Jan 9 2026; real today is March 2026 — Q1 2026 filter was wrong

6. Prompt Engineering Decisions

6.1 PTFC Architecture

The system prompt follows the PTFC framework: Person (who is asking), Task (what to do), Format (how to answer), Context (constraints and data rules). This structure was chosen because it separates concerns cleanly — the LLM knows its audience, its job, its output format, and its data rules independently.

6.2 Why 5 layers instead of a flat prompt

- Layer 0 — Domain Knowledge: Skylark-specific mappings (energy = Renewables + Powerline, not Mining). Without this, 'exclude energy' would be interpreted incorrectly.
- Layer 1 — Math Rules: Explicit formula for win rate (won / (won + dead), never won / total). LLMs default to won / total_deals which inflates win rates by including open deals.
- Layer 2 — Null Safety: Explicit rules for what to do when fields are missing. Without this, LLMs fabricate plausible-looking numbers.
- Layer 3 — Adaptive Format: 10 question types detected with matching formats (ranking → numbered list, comparison → side-by-side, trend → timeline narrative). One format for all questions produces mediocre answers.
- Layer 4 — Universal Output Rules: Applied to every answer. Bans filler phrases, enforces currency formatting, requires every answer to end with one actionable insight.
- Layer 5 — Filter Memory: Handles cumulative conversational filters across turns. Without this layer, 'what if we exclude energy' would be forgotten in the next turn.

7. Agent Trace Display

Every response includes a collapsible 'Show API Trace' expander that displays a full step-by-step execution log. This was built for transparency and debugging. The trace shows:

- Step 1: Live API fetch — both Monday.com board IDs, item counts, and fetch times
- Step 2: Data normalization — rows removed, typos fixed, quality issues found
- Step 3: Context building — what analytics were computed
- Step 4: Prompt assembly — which layers were active, which filters were injected
- Step 5: LLM response — model name, temperature, response time
- Total time breakdown — data fetch vs normalization vs LLM

Active filters are shown both in the trace and as badge labels below each answer in the chat UI, so the founder can always see what constraints are applied.

8. Challenges Faced and Solutions

Challenge	Root Cause	Solution Applied
Groq 429 rate limit (100K TPD)	Each query used ~5868 tokens — only 17 queries/day	Switched to llama-3.1-8b-instant (500K TPD) + compressed prompt from 2368 to 542 tokens
datetime.date vs float NaN crash	Monday.com API returns date strings; None becomes float NaN in DataFrame; mixed types caused >= crash	Changed <code>_parse_date()</code> to return <code>pd.Timestamp</code> (or <code>pd.NaT</code>), giving clean <code>datetime64</code> columns
'list' object has no attribute 'get'	Monday.com API returns text as list for dropdown columns in some API versions	Added <code>_safe_cv()</code> helper — checks if <code>cv</code> is dict before calling <code>.get()</code> , joins list values if needed
LLM answers starting with bare numbers	Prompt said 'lead with key number' — LLM took it literally	Added explicit GOOD/BAD examples: 'Mining is your strongest sector — 71.1%' vs '71.1% is the win rate'
Wrong quarter shown for 'this quarter'	<code>context_builder</code> used <code>deals_df.created_date.max()</code> as today — returned Jan 9, 2026 instead of March 2026	Replaced with <code>pd.Timestamp(datetime.now())</code> — uses real system clock
Emojis appearing in answers despite ban	Emoji characters in system prompt were encoded as UTF-8 and re-output by LLM	Stripped all non-ASCII characters from <code>prompt_engine.py</code> using <code>encode/decode ascii</code>

9. Time Spent

Phase	Activities	Hours
Discovery & Setup	Monday.com API exploration, column ID mapping, GraphQL query design	3 hrs

Data Layer	Normalization, date handling, financial parsing, quality detection	4 hrs
Context Builder	Sector stats, win rates, velocity, aging, weighted pipeline, quarterly trends	5 hrs
Prompt Engineering	PTFC structure, 5 layers, format detection, filter memory, example pairs	6 hrs
Bug Fixes	datetime crash, list object crash, token limit, emoji issue, quarter date bug	4 hrs
UI / UX	Streamlit layout, filter badges, trace expander, Enter-to-submit, auto-clear	2 hrs
Testing & Verification	100% accuracy check against Excel (25/25 metrics), answer quality review	3 hrs
	Total	~27 hrs

10. What I Would Do With More Time

- Streaming responses — show LLM tokens as they arrive rather than waiting for full response
- Pagination — Monday.com API has a 500 item limit; implement cursor-based pagination for boards that grow beyond this
- Chart rendering — detect when answer contains ranking or comparison data and auto-render a bar chart below the text
- Export to PDF — allow founder to export any answer with supporting data as a one-pager
- Automated accuracy testing — run 20 benchmark questions on every deploy and flag regressions
- Multi-board support — extend to support 3rd Monday.com board (e.g. HR or Finance) via the same pipeline

Summary: Every decision in this project was made with one goal: give a non-technical founder accurate, fast, trustworthy answers from their own live data — with no setup, no exports, and no spreadsheets.